Hybrid Job Search Engine Using Sparse-Dense Retrieval

Project Attribute	Final Implementation
Domain	Ad-hoc job-posting search and ranking
Dataset	1,068 structured/synthetic job postings
Sparse model	Custom BM25/BM25F-style weighted-field retrieval
Dense model	Sentence-transformer embeddings with cosine similarity
Hybrid model	Min-max score normalisation and linear score fusion
Evaluation	P@5, nDCG@10, and MRR over manually judged queries

GitHub Repository: https://github.com/oscar5198/hybrid-job-search-engine

Executive Summary

This report presents a completed Python information retrieval system for ranking job postings. The project compares a custom field-weighted BM25-style sparse retriever, a sentence-transformer dense retriever, and a hybrid model that combines normalised sparse and dense scores. The final results show that BM25 remains highly effective for skill-sensitive ranking, while hybrid retrieval improves early precision and first-relevant-result placement.

1. Introduction

Online job search is a high-volume information retrieval setting in which users submit short, open-ended queries and expect the most relevant vacancies to appear near the top of the result list. The ranking problem is difficult because job postings mix structured fields, domain-specific skill names, and free-text descriptions of responsibilities. A useful job search engine must reward exact matches for technologies such as Python, SQL, AWS, React, or Django, while also recognising that equivalent roles can be described with different vocabulary.

The implemented system addresses this challenge as an ad-hoc retrieval task over 1,068 structured/synthetic job postings. It builds three retrieval pipelines: a custom sparse retriever based on BM25-style weighted-field scoring, a dense retriever based on sentence-transformer embeddings and cosine similarity, and a hybrid retriever using linear score fusion. The report documents the final implemented project, its evaluation methodology, and the observed trade-offs between lexical and semantic ranking.

The central question is whether combining lexical matching with semantic similarity improves ranking quality for job search. The final evidence indicates a trade-off between retrieval metrics: BM25 achieves the strongest average nDCG@10, while the hybrid system achieves the best P@5 and MRR. This indicates that hybrid retrieval improves early precision, but exact field-sensitive lexical matching remains a strong baseline for job postings.

2. Background and Motivation

Job search suffers from lexical mismatch: users often describe a desired role differently from the wording used in the posting. For example, a query for backend Python API developer can be relevant to postings that emphasise server-side engineering, REST services, Django, or scalable web applications. A purely lexical system may miss these matches when the query and posting do not share enough terms.

At the same time, job search is unusually sensitive to exact terminology. If a user searches for SQL, Kubernetes, AWS, or React, the presence of that term can be a strong relevance signal. Dense semantic retrieval helps with vocabulary variation, but a general-purpose embedding model can blur distinctions between related but non-equivalent skills. This makes job search a natural setting for hybrid retrieval: lexical matching captures precise requirements, while dense similarity adds semantic flexibility.

- Lexical precision is important for role titles, technical skills, tools, and framework names.
- Semantic similarity is useful when postings and queries use different wording for similar responsibilities.
- Hybrid score fusion offers a simple, interpretable mechanism for combining both signals.

3. Related Work

Classical information retrieval models represent documents and queries through weighted lexical features. TF-IDF assigns high weight to terms that occur frequently in a document but are rare in the collection [1]. BM25 extends this tradition by modelling term frequency saturation and document-length normalisation, and it remains a competitive retrieval baseline in many practical search tasks [2]. BM25F generalises BM25 to structured documents by allowing different fields to contribute different weights [3], which is especially relevant for job postings because titles, skills, keywords, and responsibilities carry different retrieval value.

Neural dense retrieval represents queries and documents as vectors in a shared semantic space. Transformer encoders such as BERT provide the foundation for contextual language representations [4], while dual-encoder dense retrieval systems such as DPR demonstrate the use of embedding similarity for retrieval [5]. Sentence-BERT adapts transformer representations for efficient sentence and text similarity using siamese and triplet network structures [6].

Hybrid sparse-dense retrieval combines complementary strengths. Linear interpolation is a transparent fusion strategy: sparse and dense scores are normalised and combined using a weighted average. More complex methods, including late-interaction retrieval models such as ColBERT, improve token-level semantic matching at higher computational cost [7]. This project focuses on an interpretable and reproducible linear fusion baseline because it provides a clear demonstration of how sparse and dense retrieval signals contribute to ranking performance.

4. System Design

The system is organised as a Python retrieval pipeline over a CSV job-posting collection. Each posting contains structured fields including JobID, Title, ExperienceLevel, YearsOfExperience, Skills, Responsibilities, and Keywords. The retrieval models index and score the most relevant textual fields: title, skills, keywords, and responsibilities.

Component	Implemented Design
Data layer	Loads and validates the job dataset from data/job_dataset.csv using pandas.
Preprocessing	Lowercases text, removes punctuation, tokenises BM25 input, removes stopwords, and concatenates dense input fields.
Sparse retrieval	Builds a custom inverted index with field-weighted term counts and BM25-style scoring.
Dense retrieval	Encodes query and job text with all-MiniLM-L6-v2 and ranks by cosine similarity.
Hybrid retrieval	Normalises BM25 and dense scores to [0, 1], then combines them with alpha-weighted linear fusion.
Evaluation	Runs BM25, Dense, and Hybrid rankings over manually judged queries and writes metrics/plots to results/.

The hybrid model uses the following fusion equation after score normalisation:

$$\text{Hybrid} = \alpha(\text{BM25}) + (1 - \alpha)(\text{Dense})$$

The implemented default alpha is 0.5, so sparse and dense signals contribute equally after normalisation. Documents missing from one candidate list receive a zero contribution from that model in the interactive hybrid search code; the evaluation pipeline scores the full collection before fusion.

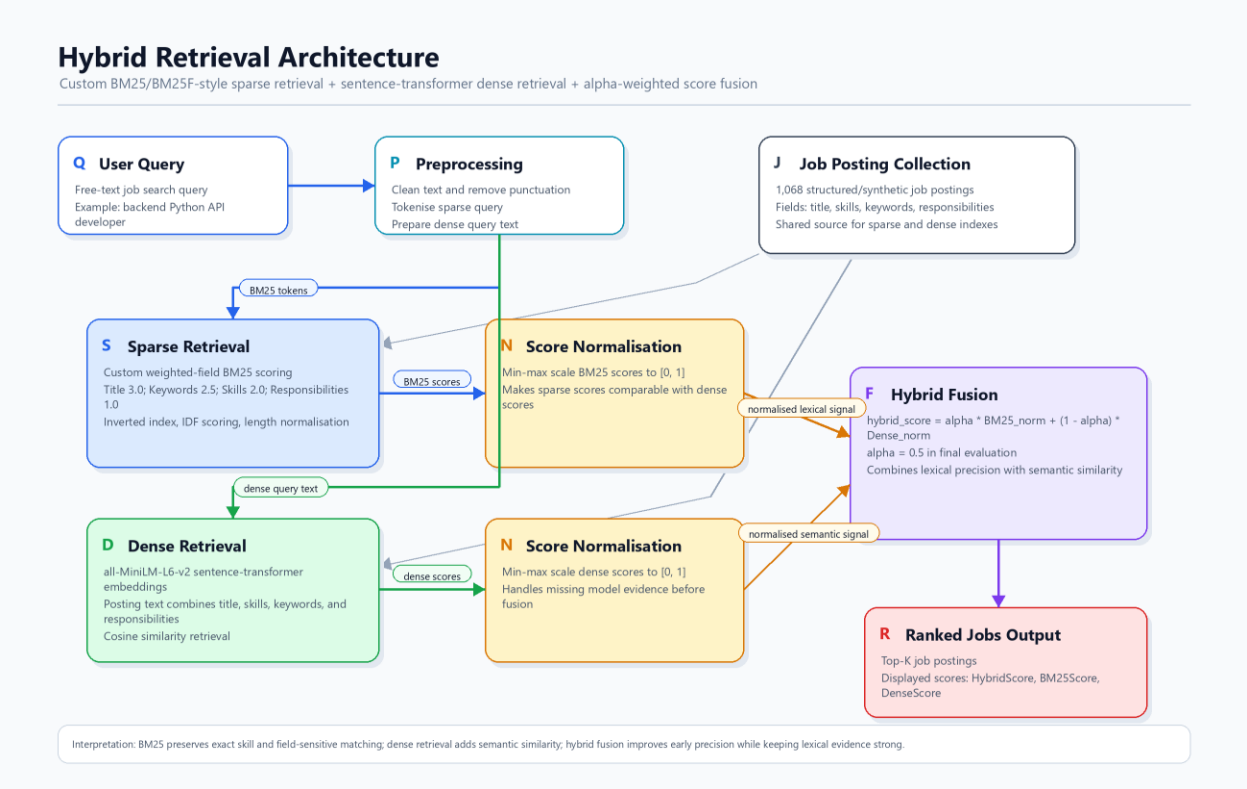

Figure 1. Hybrid Retrieval Architecture.

5. Implementation

The sparse retriever is implemented as a custom BM25Retriever class rather than through an external BM25 package. During indexing, the retriever iterates over the selected fields and accumulates weighted term frequencies. The final field weights are title = 3.0, keywords = 2.5, skills = 2.0, and responsibilities = 1.0. This weighting reflects the information value of each field: title and keywords are compact and discriminative, skills are central to job fit, and responsibilities are longer and less specific.

The dense retriever uses the sentence-transformer model all-MiniLM-L6-v2. Each posting is converted into a single dense text string by concatenating Title, Skills, Keywords, and Responsibilities. The query is cleaned and encoded with the same model, then scikit-learn cosine similarity is used to rank job embeddings. This implementation is deliberately lightweight and does not fine-tune the embedding model.

The hybrid retriever integrates the two models in src/hybrid.py. It obtains top-ranked candidates from BM25 and dense retrieval, min-max normalises their scores, and applies the alpha-weighted fusion formula. The interactive search interface returns ranked rows with HybridScore, BM25Score, and DenseScore so that the contribution of each model remains inspectable.

File/Folder	Role in Final Repository
src/preprocessing.py	Dataset loading, BM25 tokenisation, stopword handling, dense text construction, and query preprocessing.
src/bm25.py	Custom weighted-field sparse index, IDF calculation, BM25-style scoring, ranking, and score explanation.
src/dense.py	Sentence-transformer loading, embedding construction, cosine similarity retrieval, and dense result printing.
src/hybrid.py	Hybrid search engine class, score normalisation, linear fusion, and interactive hybrid ranking.
evaluation/	Queries, relevance judgements, metric functions, pooled judgements, and evaluation runner.
results/	Evaluation metrics, average result table, query examples, summary, and generated visualisations.

6. Experimental Setup

The evaluation compares BM25, Dense, and Hybrid retrieval over a controlled set of 10 manually judged job-search queries. Relevance judgements are binary and stored in the evaluation data. The evaluation script ranks the top 20 results for each model and computes three standard information retrieval metrics: Precision at 5, nDCG at 10, and Mean Reciprocal Rank.

Metric	Purpose
P@5	Measures how many of the top five returned postings are relevant, matching the user-facing importance of early results.
nDCG@10	Rewards relevant postings appearing near the top ten positions and penalises relevant results placed too low.
MRR	Measures how quickly the first relevant posting appears in the ranked list.

The dense model is fixed to all-MiniLM-L6-v2 and is not domain fine-tuned. Hybrid fusion uses alpha = 0.5. This setup makes the comparison reproducible and intentionally conservative: the dense model represents an off-the-shelf semantic retrieval baseline, while BM25 represents a field-aware lexical baseline tuned through interpretable field weights.

7. Results and Analysis

The following table reports the final average results from the repository. BM25 achieves the highest average nDCG@10, Hybrid achieves the highest P@5 and MRR, and Dense retrieval alone performs weakest across all three averages.

Model	P@5	nDCG@10	MRR	Primary Observation
BM25	0.680	0.873	0.770	Strongest average nDCG@10; robust exact skill matching.
Hybrid	0.700	0.870	0.920	Best P@5 and MRR; strongest early precision.
Dense	0.600	0.747	0.664	Weaker without domain-specific fine-tuning.

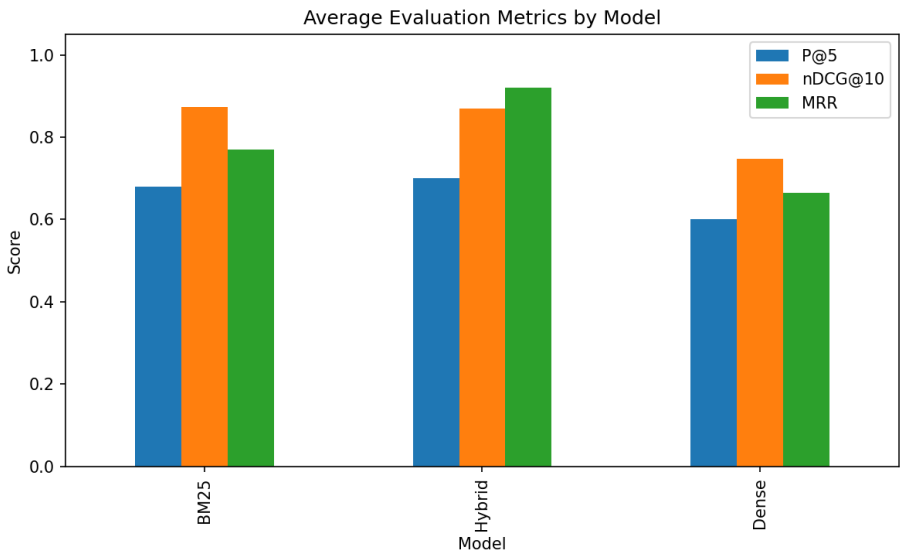

Figure 2. Average retrieval metrics by model.

The strongest interpretation is that the domain remains highly lexical. Many job-search queries include exact technologies, role names, and skill phrases. BM25 benefits from the custom weighting of title, keywords, and skills, which helps exact matches surface reliably. This explains why BM25 retains the best nDCG@10 even though Hybrid slightly improves top-five precision.

Hybrid retrieval improves P@5 from 0.680 to 0.700 and MRR from 0.770 to 0.920. The MRR improvement is especially meaningful: hybrid fusion more often places a relevant posting at the first few ranks, even when BM25 still distributes relevant results effectively throughout the top ten. This supports the motivation that semantic similarity can improve early ranking when wording differs between query and posting.

Dense retrieval alone is weaker because all-MiniLM-L6-v2 is a general-purpose sentence-transformer rather than a model fine-tuned for recruitment or skill matching. It captures broad semantic relationships, but it can underweight exact technical requirements. In job search, semantic relatedness is not always enough: a posting for Java may not be a good match for a user searching for JavaScript, even though the terms are superficially related.

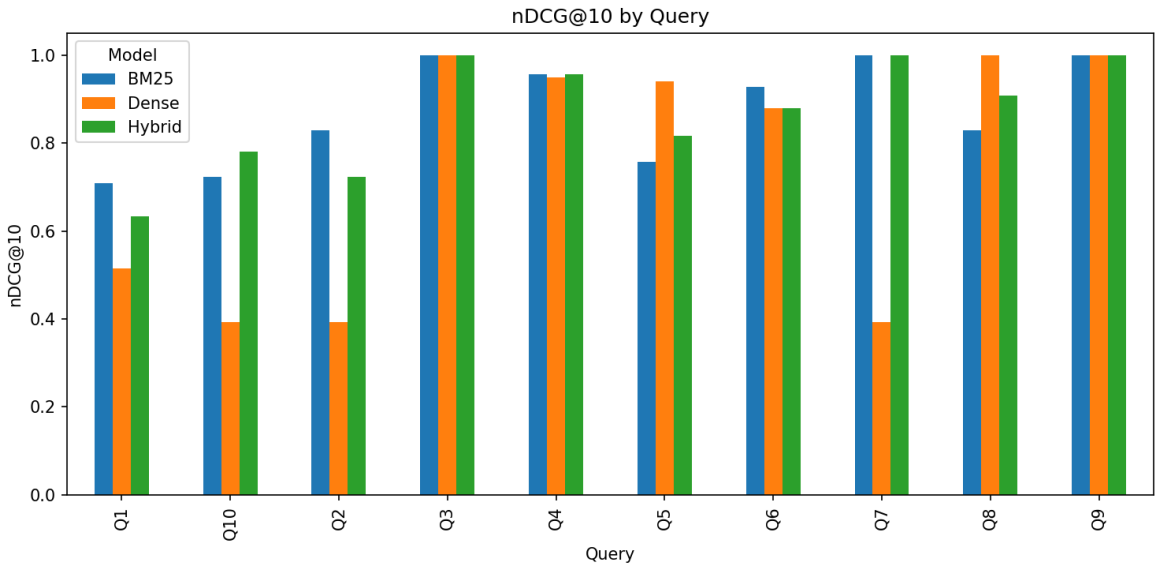

Figure 3. nDCG@10 by query across BM25, Dense, and Hybrid retrieval.

The per-query pattern is also consistent with the project summary generated by the evaluation script: BM25 performs best on most queries, Dense performs best on a smaller subset where semantic variation is more important, and Hybrid performs best where both exact skills and semantic evidence contribute useful ranking signals.

8. Limitations

- The evaluation uses a small manually judged query set, so the scores should be treated as directional rather than definitive.
- The dataset is structured/synthetic and may not fully reflect the noise, duplication, missing fields, and inconsistent formatting of live job boards.
- Relevance judgements are binary and do not capture degrees of fit such as seniority, location, salary, industry, visa constraints, or remote preference.
- The dense retriever uses all-MiniLM-L6-v2 without domain fine-tuning, so semantic ranking is limited by a general-purpose embedding space.
- Some source JobID values are reused across job families, which can make ID-only evaluation less precise.
- The hybrid fusion weight is fixed at alpha = 0.5 in the final evaluation rather than tuned on a larger validation set.

9. Future Work

1. Expand the judgement set with more queries, pooled candidates, and graded relevance labels.
2. Evaluate alternative alpha values and query-adaptive fusion strategies.
3. Fine-tune the dense model on recruitment-specific query-posting pairs or skill taxonomies.

4. Add a cross-encoder reranker for the top retrieved candidates to improve final ordering.
5. Introduce a web interface with filters for seniority, location, salary, and remote work preferences.
6. Test on a larger real-world job dataset to measure robustness under noisy production-like conditions.

10. Conclusion

This project implements and evaluates a hybrid sparse-dense job search engine over 1,068 structured/synthetic job postings. The final system includes a custom BM25/BM25F-style weighted-field sparse retriever, a sentence-transformer dense retriever, and a linear hybrid fusion model. The methodology matches the final repository: custom BM25 scoring, all-MiniLM-L6-v2 embeddings, cosine similarity retrieval, min-max score normalisation, alpha = 0.5 fusion, and evaluation with P@5, nDCG@10, and MRR.

The results show that BM25 remains the strongest model for average nDCG@10, reflecting the importance of exact skill and role matching in job search. Hybrid retrieval achieves the best P@5 and MRR, demonstrating that semantic evidence can improve early precision and place relevant postings closer to the top of the ranking. Dense retrieval alone is weaker without domain fine-tuning. Overall, the project supports a practical conclusion: hybrid retrieval is useful for improving the first results users see, but strong lexical retrieval remains essential for skill-sensitive job search.

11. References

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing and Management, vol. 24, no. 5, pp. 513-523, 1988.

[2] S. E. Robertson, S. Walker, M. M. Hancock-Beaulieu and M. Gatford, "Okapi at TREC-3," Proc. Third Text REtrieval Conf. (TREC-3), pp. 109-126, 1994.

[3] S. Robertson, H. Zaragoza and M. Taylor, "Simple BM25 extension to multiple weighted fields," Proc. CIKM 2004, pp. 42-49, 2004.

[4] J. Devlin, M. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," Proc. NAACL-HLT 2019, pp. 4171-4186, 2019.

[5] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," Proc. EMNLP 2020, pp. 6769-6781, 2020.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," Proc. EMNLP-IJCNLP 2019, pp. 3982-3992, 2019.

[7] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," Proc. SIGIR 2020, pp. 39-48, 2020.